Learning to Spread Edges: Reinforcement-Learning Construction of Spatially-Coupled LDPC Codes

Yye *et al.*

***Abstract*—Spatially-coupled low-density parity-check (SC-LDPC) codes are built from a component protograph by *edge spreading*: each systematic base edge is assigned to one of $w{+}1$ coupling components $B_0, \ldots, B_w$. In standard 5G-NR-based constructions this assignment is left to a random number generator, yet we measure that this single random choice changes the frame-error rate (FER) at a fixed operating point by roughly $7\times$. We make four contributions. (C1) We show that edge spreading is a high-leverage but routinely randomized design freedom: optimizing it alone lowers FER by 5–$11\times$ and *eliminates the error floor*; the gain holds across code rates 0.5–0.9, with a waterfall-threshold gain of 0.12–0.92 dB that is largest (~ 0.9 dB) at $R \approx 0.6$ and shrinks monotonically toward high rate. (C2) We cast edge spreading as a Markov decision process (MDP) and optimize it end-to-end with policy gradient (REINFORCE), using the FER of the real encode/channel/window-decode chain as the reward. The learned 8-parameter feature policy reduces to an interpretable rule—spread the edges of the same base column across distinct components—which removes the dominant population of 4-cycles (girth $4 \to 6$, $n_4 \approx 930 \to 0$); we state a combinatorial lemma linking same-column co-assignment to countable 4-cycles in the lifted quasi-cyclic graph, and outline how finite-length protograph EXIT (PEXIT) analysis explains the rate dependence of the waterfall gain via threshold saturation. (C3) We give a deployable criterion for *when* reinforcement learning helps: its sample-efficiency advantage over blind random search grows monotonically with per-evaluation cost and search-space size $(w{+}1)^E$—RL wins $13/15$ long-code cells (advantage growing with w), while under a cheap surrogate it merely ties. (C4) The feature policy transfers *zero-shot* to new lifting sizes and rates, a deployment property that random search and the cross-entropy method (CEM) cannot offer.**

***Index Terms*—Spatially-coupled LDPC codes, edge spreading, code construction, reinforcement learning, policy gradient, 5G NR, error floor, threshold saturation, quasi-cyclic codes.**

I. Introduction

SPATIALLY-coupled LDPC (SC-LDPC) codes couple a chain of otherwise independent LDPC component codes along a spatial dimension; under belief-propagation (BP) decoding of a terminated chain, their threshold *saturates* to the maximum-a-posteriori (MAP) threshold of the underlying component ensemble [1], [2]. This makes SC-LDPC a strong candidate for low-latency, streaming, and 6G channel coding, and the 5G-NR quasi-cyclic (QC) base graphs of 3GPP TS 38.212 [3] provide a standardized, efficiently encodable starting point.

An SC-LDPC code is obtained from a component protograph by *edge spreading*: writing the component base matrix as $B = B_0 + B_1 + \cdots + B_w$, each base edge is assigned to exactly one component B_a, and the variable position t connects through B_a to check position $t{+}a$ [4]. Different edge spreadings are known to affect *both* the iterative BP performance and the distance/cycle properties of the resulting code [4]. Yet in practice—and in the original version of the open-source pipeline used here—this assignment is simply drawn from a random number generator. The single line `rng.integers(0,w+1,size=E)` is, in effect, an unexamined design decision.

How much does that decision matter? On a fixed channel realization (i.e. using common random numbers so that only the construction changes), we measure the FER of 14 *random* edge spreadings at one operating point and observe a spread of 0.106 to 0.725—about a $7\times$ swing in FER from this one random choice alone (Section VII-A). The construction space is enormous: $(w{+}1)^E$ with E systematic base edges, e.g. $3^{40} \approx 10^{19}$ for the deep-dive configuration and as large as 31^{121} for the wide-memory sweeps. Edge spreading is therefore a high-leverage but routinely randomized construction freedom.

Contributions. We position this work as a finite-length *coding-engineering* paper in which a learning method is the tool and the code is the subject. Concretely:

- **C1 (the hard result).** Optimizing edge spreading—and nothing else—lowers FER by 5–$11\times$ and *eliminates the error floor*; the gain holds over rates 0.5–0.9, with a waterfall-threshold gain of 0.12–0.92 dB, largest at $R \approx 0.6$ and shrinking toward high rate (Sections VII-B, VII-E).
- **C2 (mechanism and theory).** We cast construction as an MDP and optimize it end-to-end by policy gradient; the learned 8-parameter policy is an interpretable "spread same-column edges" rule that removes 4-cycles. We state a combinatorial lemma connecting same-column co-assignment to countable 4-cycles in the lifted QC graph (Section V) and outline a finite-length PEXIT objective that explains the rate dependence of the gain via threshold saturation (Section IV).
- **C3 (a methodological criterion).** The sample-efficiency advantage of RL over random search/CEM grows monotonically with per-evaluation cost and search-space size $(w{+}1)^E$: RL wins $13/15$ long-code cells with the advantage growing in w, but ties under a cheap surrogate (Sections VII-F, VII-G, VII-H). This is a deployable "when to use RL" criterion.
- **C4 (a deployment property).** The feature policy transfers zero-shot to new lifting sizes and rates—something random search/CEM, which produce a code-specific as-

[TODO: author names, affiliation, e-mail, and funding acknowledgement.]
Manuscript drafted June 2026.

TABLE I
DISTINCTION FROM THE CLOSEST WORKS.

	AI Coding [6]	Esfahanizadeh / Battaglioni	**This work**
Code	block/polar	SC-LDPC	SC-LDPC
Object	gen./parity	partition matrix	**edge-spread assign**
Method	RL / GA	greedy/exhaustive/MCMC	**policy-gradient MDP**
Reward	dist./thr./BER	asy. thr. + cycles	**end-to-end FER**
Transfer	—	no (re-search)	**yes (zero-shot)**

signment vector, cannot do (Section VII-I).

The remainder follows the structure: Section II reviews SC-LDPC and edge spreading; Section III casts construction as an MDP; Section IV develops the threshold (PEXIT) objective; Section V states the combinatorial lemma; Section VI describes baselines and setup; Section VII reports results; Section VIII discusses limitations, including a scoped negative result on the error floor; and Section IX concludes.

A. Related Work and Distinction

We organize prior art along two axes: decoding versus construction, and block versus spatially-coupled codes.

*RL for LDPC **decoding*** is mature: RELDEC casts cluster scheduling as an MDP and learns sequential schedules for 5G-NR LDPC [5]; our companion line learns sliding-window scheduling for SC-LDPC. In all of these the code is *given*.

*AI/RL for **code construction*** exists but not for spatial coupling. "AI Coding" [6] proposes a constructor–evaluator framework (RL or a genetic algorithm) for *linear block / polar* codes; a deep-RL/MCTS approach learns PEG-style edge growth for LDPC *block* codes [7]. Neither targets SC-LDPC nor edge spreading.

*SC-LDPC **construction*** has been studied with non-learning search: Mitchell–Lentmaier–Costello define edge spreading and quantify its effect [4]; high-rate array-based edge spreading is designed in [8]; greedy elimination of harmful objects is used in [9]; the partition matrix is optimized by exhaustive traversal with pruning in [10], [11], whose authors note that such discrete optimization "struggles as more degrees of freedom appear"; a 6G edge-spreading Raptor-like construction [12] and an MCMC sampler [13] are likewise non-RL. After an extensive literature check, *no published work casts SC-LDPC construction / edge spreading as an MDP and optimizes it with reinforcement learning*; existing methods are greedy, exhaustive, graph-theoretic, or MCMC. Our methodological template is the neural combinatorial optimization framework of Bello *et al.* [14] with the REINFORCE estimator [15]. Table I summarizes the distinction from the closest works.

II. BACKGROUND

A. From Block Codes to a Coupled Chain

An LDPC block code is defined by a fixed parity-check matrix H (Tanner graph). An SC-LDPC code couples a chain of LDPC component codes—each described by a protograph / base matrix B of size $m_b \times n_b$—along the spatial dimension. First introduced as LDPC convolutional codes [2], the protograph view was systematized by [1].

B. Edge Spreading and the Band-Diagonal Matrix

Given coupling memory w, edge spreading splits B into $w{+}1$ component matrices,

$$B = B_0 + B_1 + \cdots + B_w, \tag{1}$$

with each base edge assigned to exactly one component. Staggering the components along the diagonal yields a band-diagonal coupled parity-check matrix with block structure

$$H_{\mathrm{SC}}[i,t] = B_{i-t} \quad \text{for } 0 \le i - t \le w, \tag{2}$$

where $t = 0, \ldots, L{-}1$ indexes variable (column) positions and $i = 0, \ldots, L{+}w{-}1$ indexes check (row) positions over a coupling length L. Variable position t thus connects through $B_0, \ldots, B_w$ to check positions $t, \ldots, t{+}w$.

C. Termination and Rate Loss

The w extra check positions at the tail are the *termination*: they impose stronger constraints at the chain boundaries (lower-degree, more reliable checks), at the cost of a small rate loss. The coupled rate R_L satisfies $R_L \to R$ as $L \to \infty$, with rate loss of order w/L.

D. Threshold Saturation

The central theoretical property of SC-LDPC is threshold saturation [1]: the BP threshold of a terminated SC-LDPC ensemble saturates to the MAP threshold of its component ensemble. Termination triggers a *decoding wave* that propagates reliability inward from the boundaries; this is the mechanism by which suboptimal iterative decoding is lifted, essentially for free, to near-MAP performance. Sliding-window decoding [16] exploits the locality of this wave: BP is run only inside a window of W positions, so latency and storage scale with W rather than L. We use $W = w{+}1$ throughout.

E. 5G-NR Systematic Edge Spreading

We build SC-LDPC from the 5G-NR QC-LDPC base graphs of 3GPP TS 38.212 [3] (BG1: 46×68, $K_b{=}22$; BG2: $42{\times}52$, $K_b{=}10$). To preserve the efficiently encodable double-diagonal / accumulator parity structure of 5G NR, we spread only the *systematic* base edges (the first K_b columns) across $B_0, \ldots, B_w$ and keep the entire parity part in B_0. Each position is then encoded by the standard 5G recursion, and the coupled $H_{\mathrm{SC}}c = 0$ holds exactly. The set of E systematic base edges is the construction variable; the parity structure and QC lifting shifts are fixed. Why edge spreading is a natural fit for sequential optimization: it is an inherently sequential assignment process (each edge in turn picks a component), its quality is governed by the cycle structure of the lifted graph, and that structure is highly sensitive to each edge's assignment—precisely where a policy gradient can climb a structural gradient while blind search cannot efficiently cover $(w{+}1)^E$.

III. Construction as a Markov Decision Process

We construct a code edge-by-edge in a fixed row-major order $e_0, \ldots, e_{E-1}$ and cast it as an MDP.

A. State, Action, Reward

- **Episode:** construct one full code (assign all E edges) and evaluate it end-to-end.
- **Time step t:** choose a component $a_t \in \{0, \ldots, w\}$ for edge e_t.
- **State s_t:** features of edge e_t under the *current partial construction*. For each candidate component a we compute the fraction of edges in the same base *row* already assigned to a (`row_frac`), the same fraction by base *column* (`col_frac`), the *global* load fraction of a (`glob_frac`), and binary indicators of whether a is still empty for this row / column (`row_empty`, `col_empty`). These running counts make the process a genuinely sequential MDP and let the policy learn structural rules such as "spread same-column edges."
- **Action a_t:** the component index.
- **Reward:** intermediate rewards are 0; at the end of the episode a terminal reward $R = -\text{FER}$ is given, obtained by placing the constructed code into the *real* encode $\rightarrow$ BPSK+AWGN $\rightarrow$ sliding-window BP $\rightarrow$ FER chain at the training SNR over a batch of common-random-number (CRN) frames. The ground truth is used only to score a constructed code at design time; the learned construction (the assignment vector) carries no ground truth and is directly deployable, exactly as for any code design.

B. Policy Gradient

Maximizing $\mathbb{E}[R]$ over the factorized policy $\pi_\theta(a) = \prod_t \pi(a_t \mid s_t)$ is REINFORCE [15], [14]:

$$\nabla J = \mathbb{E}\left[(R - b)\sum_t \nabla \log \pi(a_t \mid s_t)\right], \tag{3}$$

where b is an *in-batch baseline* (the mean of R over the batch). On top of this we use *common random numbers*: every candidate code in a batch is scored on the *same* frames, so the advantage $R_i - b$ has very low variance and the policy learns stably even when each code is scored on only a few frames. This variance reduction is what makes the method tractable on CPU. We optimize with Adam (pure NumPy), standardize advantages within a batch, and add an entropy bonus to encourage exploration. Both policy gradients are verified by finite differences.

C. Two Policies

1) **Feature policy (primary).** A linear softmax over the per-component features, with parameters $w{+}1$ component biases plus a handful of shared weights—*8 numbers in total for w=2, independent of the code size.* Hence the learned policy can transfer zero-shot to codes with different Z/L/rate (provided w matches), and it is interpretable.
2) **Per-edge policy (ablation).** Independent logits per edge ($E \times C$ parameters): maximally expressive but non-transferable, used as a control for whether a generalizing structure is needed.

D. Algorithmic Notes

The implementation is pure NumPy with multiprocessing over CPU cores; the masks (puncturing/termination/transmitted positions) depend only on L, w, K_b, n_b, Z and *not* on the assignment, so all candidate codes can be scored on bit-identical information and noise (CRN), which the evaluator is tested to reproduce bit-exactly. The champion of each method is re-tested on an *independent*, larger frame bank to remove selection noise.

IV. A Threshold (PEXIT) Objective

This section develops a *frame-free, principled* objective and predictor that (a) addresses the "Monte-Carlo only" concern, (b) *explains* the rate dependence of the waterfall gain observed in C1, and (c) is the sharpest instance of the C3 "expensive evaluation $\Rightarrow$ RL wins" criterion.

A. Why Asymptotic Thresholds Are Nearly Construction-Invariant

By threshold saturation (Section II-D), the asymptotic BP threshold of a good terminated SC-LDPC ensemble approaches the component MAP threshold, *independently of fine construction detail*. This is precisely why the asymptotic threshold cannot, on its own, separate two good edge spreadings: the BP–MAP gap that coupling fills is large at low rate (strong component, many checks per position) and small at high rate (weak component). Consequently the *waterfall* construction gain is largest at moderate rate and shrinks toward high rate—exactly the trend measured in Section VII-E. What *does* separate finite-length constructions is the finite-length PEXIT recursion including the termination boundary, together with the cycle structure.

B. Protograph EXIT for the Coupled Graph

We adopt protograph EXIT (PEXIT) [17] over the band-diagonal protograph defined by the assignment (components $B_0, \ldots, B_w$, coupling length L, and the termination boundary), using the BIAWGN J/J^{-1} mutual-information transfer functions and the finite-length protograph analysis of [18]. The recursion returns (i) the E_b/N_0 threshold required for convergence and (ii) a finite-length PEXIT BER proxy. As a reward this is plugged into the same pipeline via $R = -\text{threshold}_{E_b/N_0}$, reusing the REINFORCE machinery unchanged and replacing the Monte-Carlo evaluator with a PEXIT evaluator.

C. Honest Scope

We do *not* claim that an asymptotic PEXIT threshold strongly discriminates constructions—threshold saturation makes the asymptotic thresholds of good constructions converge, which is itself the theory to be told. The discriminating

TABLE II
PEXIT VALIDATION (PLACEHOLDER—EXPERIMENT PENDING).

Quantity	Value
Spearman(ρ): PEXIT thr. vs MC waterfall thr.	[TODO: pending]
PEXIT-reward champion FER (MC re-test)	[TODO: pending]
MC-reward champion FER (MC re-test)	[TODO: pending]
PEXIT eval cost / MC eval cost	[TODO: pending]

quantity is the *finite-length* PEXIT recursion (with termination) plus the cycle structure. The deliverable of this section is therefore an *explanation* and a *principled predictor*, not an overwhelming new reward.

D. Validation Experiments (Pending)

The PEXIT objective is specified but the numerical validation is not yet run. Three experiments are planned:

1) Spearman correlation between the PEXIT threshold ordering and the Monte-Carlo waterfall-threshold ordering, to establish the proxy's validity. [TODO: run PEXIT-vs-MC Spearman experiment; see TCOM_PLAN Sec.5(1).]
2) Train RL with the PEXIT reward, then re-test the champion by Monte-Carlo and compare against the MC-reward champion. [TODO: run PEXIT-reward RL training + MC re-test.]
3) Measure the PEXIT evaluation cost versus the Monte-Carlo cost to reinforce C3. [TODO: measure PEXIT vs MC evaluation cost.]

Numerical results are deferred; the placeholder table is Table II.

V. A COMBINATORIAL LEMMA: SAME-COLUMN CO-ASSIGNMENT VS. 4-CYCLES

We now make precise the rule that the feature policy learns (Section VII-D), turning the empirical observation "spread same-column edges" into a statement about 4-cycles in the lifted QC graph.

A. Lemma Statement

Lemma (informal). Let an SC-LDPC code be obtained from a 5G-NR base graph by edge spreading according to an assignment, and then QC-lifted by circulant shifts P^s ($Z \times Z$ circulants). Suppose two systematic base edges e_i, e_j of the same variable base column are assigned to the same component B_a and share the same check base row. Then, after lifting, the pair produces a countable number of length-4 cycles between the corresponding (variable block, check block) pair, governed by the standard QC 4-cycle condition [19]. Assigning e_i, e_j to distinct components $B_a \neq B_b$ places them at *different* check-position blocks of the coupled graph, breaking that 4-cycle. Consequently, a construction that maximizes within-column component diversity drives the per-column 4-cycle count to a lower bound of zero.

B. Proof Sketch

A length-4 cycle in a QC Tanner graph passes through two variable blocks and two check blocks connected by four circulant edges. For two systematic edges in the same base column assigned to the same component and incident to the same check base row, the lifted circulants are co-located in the same block of H_{SC}; the standard QC condition of [19] then counts the resulting short cycles through the differences of the circulant shifts (a gcd-type count), which is generically positive. Spreading the two edges to different components $B_a \neq B_b$ relocates one of them to a different check-position block ($t+a$ vs. $t+b$) of the coupled band-diagonal structure, so the two edges no longer share both endpoints' blocks and the 4-cycle is destroyed. This is the same selective-cycle-avoidance principle formalized by the ACE metric [20], here specialized to the coupled edge-spreading degree of freedom. Maximizing within-column component diversity therefore minimizes the count of these dominant same-column 4-cycles.

C. Numerical Validation (Pending)

The lemma's prediction is to be cross-checked against the measured 4-cycle counts using the repository's `count_4cycles` and `girth` routines, comparing the round-robin / seed-0 counts (≈ 930) to the optimized count (0), and verifying the scaling across $Z \in \{16, 32, 64\}$. [TODO: run numerical lemma check (count_4cycles cross-check, measured 930 $\rightarrow$ 0, across Z).] This numerical confirmation is consistent with the mechanism measurements in Section VII-D, but the dedicated lemma-validation script has not yet been run.

VI. BASELINES AND EXPERIMENTAL SETUP

A. Baselines

We compare the two RL policies against four baselines under identical evaluator, budget, and validation:

- **Random search** (key baseline): sample a batch of uniformly random assignments per step and keep the best by validation FER.
- **Cross-entropy method (CEM):** a strong distribution-based optimizer maintaining per-edge categorical distributions and updating from elites.
- **Round-robin:** a naive perfectly-balanced heuristic.
- **Repository default (seed-0):** the single random assignment shipped by the original pipeline.
- **Uncoupled component code:** the 5G-NR component code with no coupling.

All four optimizers (RL-feature, RL-per-edge, CEM, random search) share the same evaluator, the same evaluation budget (steps $\times$ batch), and the same validation protocol (fixed validation frames, then a high-precision re-test of the champion on a large independent frame bank). Any difference is attributable to the search strategy alone.

B. Configuration

The deep-dive operating point is BG2, i_{LS}=0, Z=16, m_p=8 (component rate $\approx$ 0.625, coupled $R \approx$ 0.603), L=30,

TABLE III
CONSTRUCTION DECIDES SUCCESS: 2000-FRAME RE-TEST AT 2.5 DB (BG2, Z=16, L=30, w=2).

Construction	FER	BER	n_4	girth
CEM	**0.064**	1.37×10^{-4}	0	6
RL per-edge policy	0.080	1.98×10^{-4}	0	6
RL feature policy	0.092	2.67×10^{-4}	0	6
RL feature (greedy)	0.112	3.47×10^{-4}	0	6
Random search (budget 640)	0.133	2.60×10^{-4}	0	6
Round-robin	0.661	1.18×10^{-3}	944	4
Default seed-0	0.698	1.10×10^{-3}	928	4

w=2, sliding window W=6, normalized min-sum with α=0.8, training SNR 2.5 dB, E=40. Each optimizer uses an equal budget of $32 \times 20 = 640$ construction evaluations at 36 CRN frames each; the champion is re-tested on an independent 2000-frame bank. Rate-sweep, long-code, and wide-memory configurations are detailed at the corresponding results subsections. Experiments were run on CPU (a workstation for the deep-dive, and 64–200 core cluster pods for the sweeps).

VII. RESULTS

A. Headroom: Construction Alone Spans 7×

Using the CRN probe (same frames, only the construction changes) we measure the FER of 14 random edge spreadings at the deep-dive operating point: the FER spans 0.106 to 0.725, about a 7× swing (BER spans ≈8.5×). This confirms (a) that the construction carries large headroom and (b) that 2.5 dB is an operating point with clear, non-saturated ordering—a good training point.

B. Construction Decides Success or Failure

Table III reports the high-precision 2000-frame re-test at 2.5 dB (lower FER is better), together with lifted-graph 4-cycle count n_4 and girth. Optimizing the single edge-spreading knob lowers FER from 0.70 to 0.06–0.13—about 5–11×. The repository default (seed-0) is among the *worst* constructions (0.698), and so is the naive perfectly-balanced round-robin heuristic (0.66).

The more important effect is on the *error floor*. Table IV and Fig. 1 give the BER-vs-E_b/N_0 waterfall: optimized constructions have *no* error floor (decaying to $\sim10^{-6}/0$ at high SNR), whereas the default/naive constructions exhibit a floor at $\approx$ 3–6 $\times$ 10^{-5}. Measured at the BER= 10^{-3} waterfall threshold, the optimized constructions sit at 2.30–2.38 dB, the default/naive at 2.53–2.54 dB ($\sim$ 0.16 dB construction gain), and the uncoupled component code at 3.11 dB ($\sim$ 0.75 dB coupling gain). The construction dividend is therefore largest in the low-BER / error-floor region, consistent with the theory that short cycles mainly hurt the floor.

C. Sample Efficiency at Fixed Budget

Fig. 2 and Table V show the best validation FER versus number of evaluations. Random search stalls at $\approx$ 0.09 after about 140 evaluations (best-of-k is an order statistic with a very slow tail), while RL/CEM keep descending to 0.035–0.06 via a structured policy. On the 2000-frame re-test, RL (0.080/0.092) and CEM (0.064) both consistently beat random search (0.133), and even the *deterministic greedy* rule (0.112)—which contains no sampling luck—beats random search.

TABLE IV
BER-VS-E_b/N_0 WATERFALL (DEEP-DIVE OPERATING POINT). "FLOOR" MARKS THE HIGH-SNR SATURATION OF THE DEFAULT/NAIVE CONSTRUCTIONS.

E_b/N_0 (dB)	2.2	2.5	2.8	3.1	3.4
RL feature	5.5e−3	3.4e−4	1.9e−5	5.1e−6	0
CEM	2.7e−3	1.4e−4	4.7e−6	0	0
Random search	3.9e−3	3.7e−4	3.7e−5	2.7e−6	0
Seed-0	6.2e−3	1.2e−3	3.5e−4	1.4e−4	6.4e−5†
Round-robin	6.8e−3	1.1e−3	2.6e−4	1.0e−4	3.3e−5†
Uncoupled	2.3e−2	9.2e−3	3.1e−3	1.0e−3	1.9e−4

† error floor.

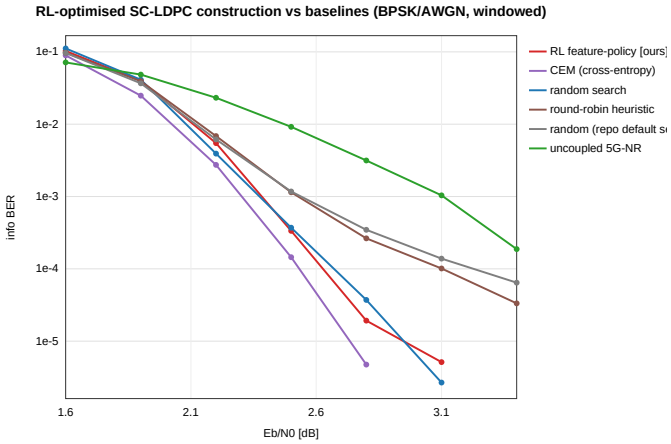

Fig. 1. BER-vs-E_b/N_0 waterfall for six constructions. Optimized constructions (RL/CEM/random search) decay without a floor; the repository default (seed-0) and the naive round-robin heuristic exhibit an error floor at $\approx$ 3–6 $\times$ 10^{-5}; the uncoupled component code is worst.

D. Mechanism: RL Rediscovers "Avoid 4-Cycles"

All optimized constructions have $n_4 = 0$ and girth 6; the default/naive constructions have $n_4 \approx 930$ (round-robin 944, seed-0 928) and girth 4 (random reference: mean $n_4 = 525.6$, range $[0, 1408]$ over 40 samples). It is precisely these $\approx$930 4-cycles that cause the floor of the default/naive constructions; optimization clears the 4-cycles and the floor disappears. The learned 8-dimensional feature weights are listed in Table VI. Read out, they encode an interpretable rule: *spread the systematic edges of the same base column across distinct components and keep global load balanced* (strongly negative col-frac / glob-frac weights, strongly positive col-empty weight). This is exactly the same-column 4-cycle avoidance of Section V, rediscovered *from the end-to-end FER reward alone*, with no hand-coded cycle / trapping-set knowledge.

TABLE V
BEST VALIDATION FER VS. NUMBER OF EVALUATIONS (LEARNING CURVE).

Evaluations	∼140	∼260	∼380	∼500	∼620
Random search	0.09	0.09	0.09	0.09	**0.09**
RL feature	0.09	0.09	0.075	0.065	**0.060**
RL per-edge	0.09	0.075	0.07	0.07	**0.035**
CEM	0.09	0.075	0.05	0.05	**0.050**

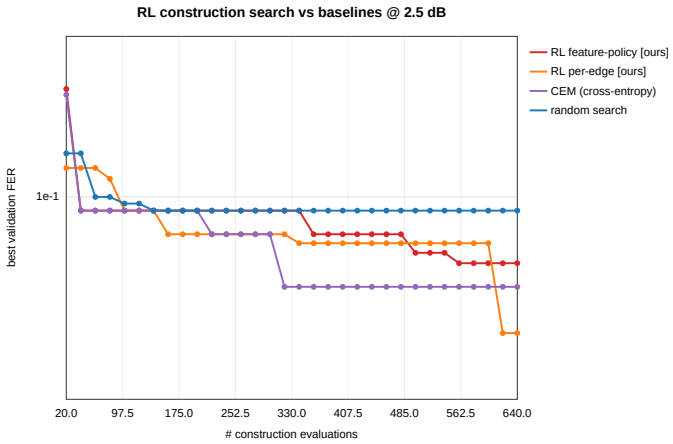

Fig. 2. Best validation FER vs. number of construction evaluations. Random search stalls at ≈ 0.09; RL (both policies) and CEM keep descending via a structured policy.

E. *Full-Rate Sweep* 0.5–0.9

We sweep the optimization across the rate range (low rate on BG2, high rate on BG1), auto-selecting the operating SNR per rate (typical random construction at FER≈ 0.3), with budget $64 \times 18 = 1152$ evaluations per optimizer. Table VII reports FER at the operating point and the waterfall-threshold gain at FER= 0.1 (seed-0 minus best optimizer). Three findings: *(i)* all optimizers beat the seed-0 default and the naive round-robin at *every* rate (0.005–0.39 vs. baseline 0.31–0.71), with a waterfall gain of 0.12–0.92 dB that is largest at moderate rate ($R \approx 0.6$, $\sim$0.9 dB) and shrinks monotonically toward high rate (Fig. 3); this matches the threshold-saturation theory of Section IV. *(ii)* At this large budget the four optimizers are comparable—best-of-1152 random search is itself a strong baseline that erases RL's sample-efficiency edge. *(iii)* The mechanism migrates with rate: at low/moderate rate the gain is driven by avoiding 4-cycles, while at high rate 4-cycles decouple from performance (e.g. a round-robin code with $n_4 = 3264$ is not bad at $R = 0.7$) and the dominant harmful structure is no longer the 4-cycle.

F. *The RL Home Court: Long Codes and Large* w

We now test the central thesis—RL's advantage appears when evaluation is *expensive* and the space is *large*. We move to long 5G-NR BG1 codes (Z=32, component codewords ≈ 860–1470 bits), high wireless rates $\{1/2, 2/3, 3/4, 5/6, 7/8\}$, and coupling memory $w \in \{1, 2, 3\}$, on 2×80 core cluster pods. A single frame costs $\sim$ 0.9 s, so each method gets only a small budget of $\sim$640 evaluations—sample efficiency becomes the hard currency—and the search space $(w{+}1)^E$ (E=69–172) reaches 4^{172} at w=3, far beyond what random search can cover. Table VIII gives FER at the operating point.

TABLE VI
LEARNED FEATURE-POLICY WEIGHTS (THE CONSTRUCTION RULE).

bias B_0	bias B_1	bias B_2	row-frac	**col-frac**	**glob-frac**	row-empty	**col-empty**
+0.33	+0.28	−0.12	−0.49	**−1.85**	**−2.30**	+0.69	**+1.61**

TABLE VII
FULL-RATE SWEEP: FER AT THE OPERATING POINT (1152 EVALS/OPTIMIZER) AND WATERFALL-THRESHOLD GAIN AT FER= 0.1.

R	BG	RL-f	RL-pe	CEM	rand	RR	seed0	gain
0.479	2	0.156	0.109	0.110	**0.091**	0.625	0.517	0.49 dB
0.603	2	0.098	0.112	0.098	**0.095**	0.644	0.709	**0.92 dB**
0.692	1	0.008	0.007	0.007	**0.005**	0.012	0.074	0.17 dB
0.770	1	0.360	0.385	**0.279**	0.386	0.499	0.696	0.14 dB
0.906	1	**0.164**	0.176	0.203	0.186	0.313	0.318	0.12 dB

Findings. *(i)* RL (feature or per-edge) beats same-budget random search on 13/15 cells—in sharp contrast to the "tie at large budget" of the mid-scale sweep. *(ii)* The advantage grows with w: w=1 → 4/5, w=2 → 4/5, w=3 → 5/5, and RL often drives FER to exactly 0 while random search stalls at 0.05–0.14 (e.g. R=2/3, w=3: RL 0.000 vs. random 0.140). *(iii)* RL dominates the repository default across the board. *(iv)* Honestly, the two exceptions (R=5/6, w=1 and R=7/8, w=2) both occur at small w where the space is small and random search suffices, and the R=1/2, w=1 auto-operating point is low (FER≈0.9, poor discrimination). Fig. 4 plots the RL/random advantage by rate and w.

G. *Pushing* w *to 12: Advantage Grows Monotonically*

We extend the expensive Monte-Carlo evaluation to $w \in \{3, 5, 8, 12\}$ (window $W = w{+}1$), full rate, 20 cells, on an eviction-resistant 100-core pod. Table IX shows the trend cleanly: as w grows from 3 to 8, random search gets steadily worse (e.g. R=2/3: 0.040 → 0.083 → 0.133) while RL gets steadily better, often reaching exactly 0 (0.017 → 0.010 → **0.000**). RL's win rate by w is w=3 : 5/5, w=5 : 3/4, w=8 : 4/4, w=12 : 2/2 (over cells with signal; the rest are ties, none losses). The default construction is almost uniformly 0.7–1.0 (fully broken). Four "no-signal" cells (where all methods give FER= 1.0) arise because the large-w rate loss pulls the SC rate too low (e.g. R=1/2, w=12 ⇒ SC 0.427) so the auto operating point falls below the waterfall—not a decoding bug; the same w=8 is fine at R=2/3.

H. *Cheap Surrogate at* $w \leq 30$*: a Clear "When to Use RL" Boundary*

End-to-end Monte-Carlo is infeasible at very large w (the window must satisfy $W \geq w{+}1$; $\sim$ 18 s/frame at w=30, L=300), so we switch the reward to a fast structural surrogate—minimize the lifted-graph 4-cycle count ($\sim$ 0.3 s, independent of w)—and sweep $w \in \{3, 5, 8, 12, 16, 20, 30\}$,

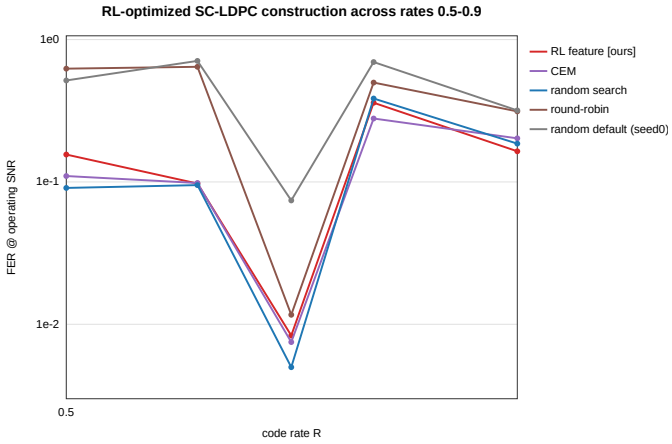

Fig. 3. Waterfall-threshold construction gain (seed-0 minus best optimizer at FER$=0.1$) vs. code rate. The gain is largest at moderate rate ($R \approx 0.6$) and shrinks toward high rate, consistent with threshold saturation closing the BP–MAP gap (Section IV).

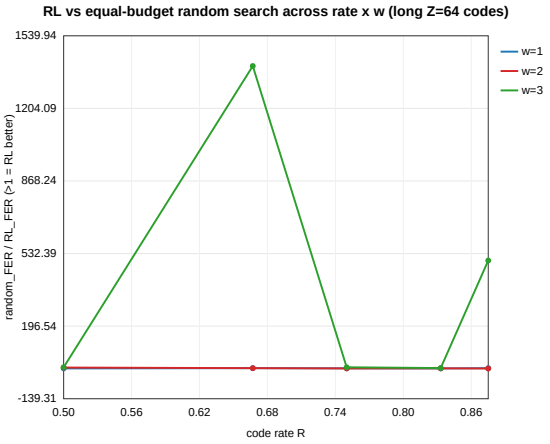

Fig. 4. RL-vs-random-search advantage across the long-code cells, by rate and coupling memory w. The advantage is systematic and grows with w (search-space size).

TABLE VIII
LONG-CODE, HIGH-RATE, LARGE-w CELLS (BG1, Z=32): FER AT THE OPERATING POINT.

R	w	E	SNR	RL-f	RL-pe	CEM	rand	seed0
1/2	1	172	1.5	0.913	0.900	0.910	0.907	0.993
1/2	2	172	2.0	**0.017**	0.043	0.073	0.083	0.707
1/2	3	172	2.5	**0.010**	0.010	0.080	0.063	0.830
2/3	1	121	2.5	**0.040**	0.057	0.083	0.080	0.127
2/3	2	121	2.5	**0.090**	0.117	0.100	0.197	0.917
2/3	3	121	3.0	**0.000**	0.010	0.017	0.140	0.773
3/4	1	96	3.0	**0.033**	0.060	0.033	0.047	0.077
3/4	2	96	3.0	0.110	0.120	**0.027**	0.120	0.670
3/4	3	96	3.5	0.003	**0.000**	0.040	0.020	0.527
5/6	1	75	3.5	0.200	0.240	0.180	**0.167**	0.430
5/6	2	75	3.5	**0.293**	0.320	0.313	0.330	0.737
5/6	3	75	4.0	**0.010**	0.037	0.067	0.027	0.400
7/8	1	69	4.0	**0.040**	0.067	0.077	0.080	0.097
7/8	2	69	4.0	0.117	0.123	0.147	**0.073**	0.290
7/8	3	69	4.5	**0.000**	0.007	0.057	0.050	0.280

$L \in \{30, \ldots, 240\}$. Table X reports n_4 at L=60. Both optimization (RL and random search) drive n_4 to 0 at every w, while the single seed-0 construction is unreliable ($\approx$900 at w=5–16). Under this *cheap* surrogate, RL $\approx$ random search—which is exactly the point: when evaluation is cheap, a brute-force best-of-1400 reaches the same n_4=0, and RL has no extra advantage. RL pays off only when the objective is *expensive* (long-code Monte-Carlo, density evolution / PEXIT). This unifies the three regimes into one rule: *RL's value rises monotonically with per-evaluation cost*.

I. Zero-Shot Transfer

We move the 8 learned feature weights *unchanged* to two new codes and greedily construct (no search on the new code). Table XI shows the zero-shot rule beats both the typical random construction (median) and the round-robin heuristic on both new configurations, indicating that RL learned a genuine structural principle rather than overfitting one code. This is a deployment property random search / CEM cannot provide, since they output a code-specific assignment that must be re-searched for every new code. (A best-of-12 random search can occasionally edge out the single zero-shot greedy code—an unequal 12-vs-0-evaluation comparison.)

TABLE IX
WIDE-MEMORY MONTE-CARLO SWEEP (RL / RANDOM-SEARCH FER). "—" DENOTES A NO-SIGNAL CELL (ALL METHODS FER$=1.0$).

R	w=3	w=5	w=8	w=12
1/2	0.000 / 0.037	—	—	—
2/3	0.017 / 0.040	0.010 / 0.083	**0.000 / 0.133**	—
3/4	0.007 / 0.033	0.010 / 0.010	**0.000 / 0.010**	0.003 / 0.087
5/6	0.007 / 0.063	0.007 / 0.143	**0.000 / 0.203**	—
7/8	0.003 / 0.100	0.003 / 0.023	**0.000 / 0.043**	0.010 / 0.037

J. Threshold (PEXIT) Objective Results

The PEXIT objective of Section IV is specified and integrated into the pipeline, but its numerical validation is still pending; see Section IV-D and the placeholder Table II. [TODO: insert PEXIT numerical results (Spearman, PEXIT-reward champion MC re-test, eval-cost ratio) once the experiment of Section IV-D is run.]

VIII. DISCUSSION AND LIMITATIONS

A. What RL Buys, Honestly

The hardest, most robust conclusion is construction-level, not RL-level: optimizing edge spreading lowers FER by 5–11$\times$ and eliminates the error floor, and the repository default is among the worst constructions. RL's specific value is threefold: it *rediscovers an interpretable rule* from the FER reward alone, it is *more sample-efficient* than same-budget random search, and its feature policy *transfers zero-shot*.

B. Conditional, Not Unconditional, Wins

RL's advantage over the baselines is *conditional*, and we present this as an honest feature rather than a weakness.

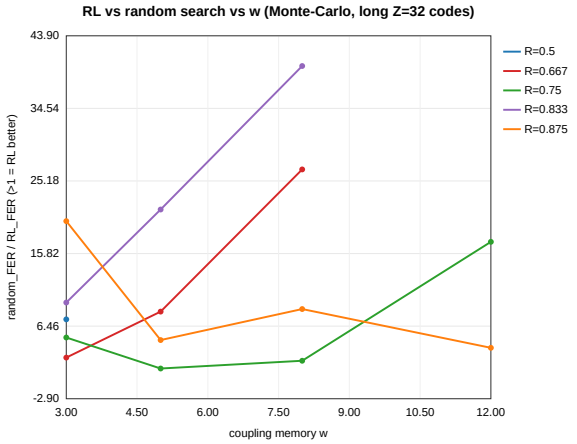

Fig. 5. Wide-memory ($w \leq 12$) Monte-Carlo sweep: RL vs. random-search FER. As w grows, random search degrades while RL improves toward 0, nailing the "advantage grows with evaluation cost $\times$ space" rule.

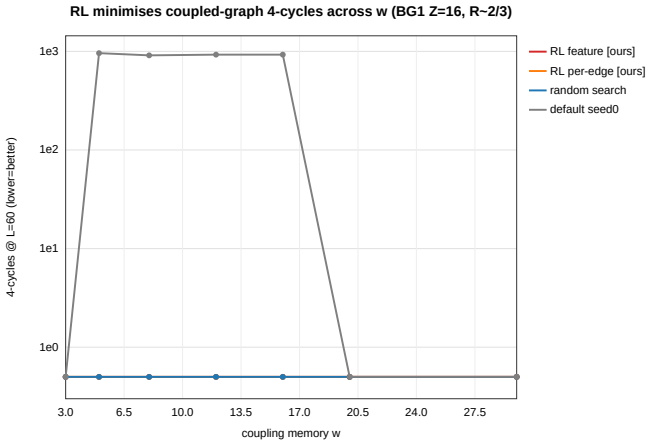

Fig. 6. n_4 at L=60 vs. coupling memory w under the cheap 4-cycle surrogate. Optimization (RL or random) reaches n_4=0 for all w; the single seed-0 construction is unreliable. Under a cheap objective, RL $\approx$ random search.

TABLE X
CHEAP 4-CYCLE SURROGATE, $w \leq 30$: n_4 AT L=60 (LOWER IS BETTER).

w	3	5	8	12	16	20	30
RL feature	0	0	0	0	0	0	0
RL per-edge	0	0	0	0	0	0	0
Random search	0	0	0	0	0	0	0
Default seed-0	0	960	912	928	928	0	0

TABLE XI
ZERO-SHOT TRANSFER OF THE 8 LEARNED WEIGHTS (GREEDY, NO RE-SEARCH): FER ON NEW CODES.

Transfer target	rate	zero-shot	random (med.)	round-robin
Z=24 (longer blocks)	0.603	**0.124**	0.146	0.240
m_p=6 (higher rate)	0.693	**0.788**	0.852	0.960

At a large evaluation budget, best-of-N random search is a strong baseline and CEM ties or slightly beats policy-gradient RL on single-point FER (e.g. CEM 0.064 vs. RL feature 0.092 in Table III); in the deep waterfall, several 4-cycle-free constructions converge within Monte-Carlo noise. The first-order factor is whether a construction avoids 4-cycles, which several optimizers can achieve; RL's incremental value is automation, interpretability, and transfer—and a clean criterion for *when* sample efficiency matters (Sections VII-F–VII-H).

C. Scale and Scope

This is an empirical finite-length study with no new asymptotic theorem: the reward is single-operating-point Monte-Carlo FER rather than a closed-form or learned objective (the PEXIT objective of Section IV is the principled remedy, with numerics pending). Experiments are mid-scale and CPU-only ($Z \leq 128$, $w \leq 30$, $L \leq 300$); larger lifting and hardware are not a contribution dimension here—indeed, the "expensive evaluation" argument is exactly why the criterion holds even at mid scale. Large-w high-rate Monte-Carlo has a calibration difficulty: coupling rate loss steepens strong-code waterfalls, so a few auto operating points fall off the waterfall (FER$=1.0$), which a more robust operating-point selection or a threshold-type objective would fix.

D. A Scoped Negative Result on the Error Floor

We tested whether a cheap absorbing-set surrogate could *directly* target the finite-length error floor, and the answer is negative—an honest, scoped result that points to future work. In an overnight campaign, the measured error floor spans about 2.5 orders of magnitude across constructions (so the floor *is* strongly construction-dependent), yet the Spearman correlation between the cheap absorbing-set count ($a \leq 6$) and the floor is only $\rho \approx -0.13$; minimizing the absorbing-set count (`cem_floor`) gave a *worse* floor (6.1×10^{-7}) than the waterfall-objective optimizer (`cem_waterfall`, 3.4×10^{-8}). Moreover, the feature policy that transferred zero-shot under the *waterfall* objective *degenerated* under the *floor* objective (collapsing to an all-zero assignment with a much larger absorbing-set count and a floor of its own). The takeaway: the standard cheap absorbing-set enumeration does *not* predict this class of finite-length floors, and minimizing it can even be harmful. A faithful floor objective therefore needs either expensive Monte-Carlo or a *learned* neural floor surrogate that amortizes the otherwise-infeasible trapping-set enumeration—the natural next direction (and outside the scope of this paper).

E. Future Work

In leverage order: (1) replace/augment the reward with a DE/PEXIT threshold (the sharpest instance of the "expensive evaluation $\Rightarrow$ RL wins" thesis); (2) complete the numerical lemma validation (Section V-C); (3) jointly optimize the QC lifting shifts (further enlarging the space and RL's home court) and meta-learn across rates/SNRs to produce code families; (4) combine orthogonally with RL decoding—learn how to *build* and how to *decode* together; and (5) the learned neural floor surrogate of Section VIII-D.

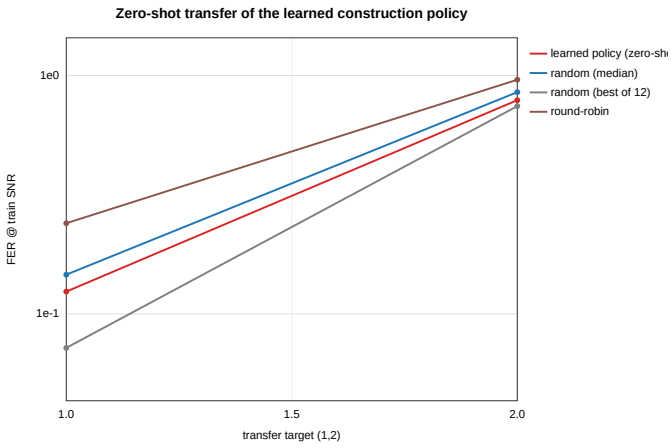

Fig. 7. Zero-shot transfer of the learned feature policy to new codes. The transferred rule (greedy, no re-search) is below the random-construction median and the round-robin heuristic on both new configurations.

IX. Conclusion

Edge spreading is a high-leverage but routinely randomized construction freedom in 5G-NR-based SC-LDPC codes. Casting it as an MDP and optimizing it end-to-end with policy gradient—using the real-chain FER as reward—lowers FER by 5–$11\times$, eliminates the error floor across rates 0.5–0.9 (waterfall gain 0.12–0.92 dB, largest at $R\approx 0.6$), and rediscovers an interpretable, zero-shot-transferable "spread same-column edges" rule whose mechanism is the elimination of 4-cycles. Beyond the codes themselves, the work contributes a deployable methodological criterion: the sample-efficiency advantage of reinforcement learning over blind search rises monotonically with per-evaluation cost and search-space size—RL is not universally superior, but in the expensive-evaluation, large-space regime of real finite-length code design it is systematically better than blind search.

References

[1] S. Kudekar, T. J. Richardson, and R. L. Urbanke, "Threshold saturation via spatial coupling: Why convolutional LDPC ensembles perform so well over the BEC," *IEEE Trans. Inf. Theory*, vol. 57, no. 2, pp. 803–834, 2011.

[2] A. J. Felström and K. S. Zigangirov, "Time-varying periodic convolutional codes with low-density parity-check matrix," *IEEE Trans. Inf. Theory*, vol. 45, no. 6, pp. 2181–2191, 1999.

[3] 3GPP, "NR; multiplexing and channel coding," 3rd Generation Partnership Project (3GPP), Tech. Rep. TS 38.212, 2018.

[4] D. G. M. Mitchell, M. Lentmaier, and J. Costello, Daniel J., "Spatially coupled LDPC codes constructed from protographs," *IEEE Trans. Inf. Theory*, vol. 61, no. 9, pp. 4866–4889, 2015.

[5] S. Habib, A. Beemer, and J. Kliewer, "RELDEC: Reinforcement learning-based decoding of moderate length LDPC codes," *IEEE Trans. Commun.*, 2023.

[6] L. Huang, H. Zhang, R. Li, Y. Ge, and J. Wang, "AI coding: Learning to construct error correction codes," *IEEE Trans. Commun.*, vol. 68, no. 1, pp. 26–39, 2020.

[7] M. Zhang, Q. Huang, S. Wang, and Z. Wang, "Construction of LDPC codes based on deep reinforcement learning," in *Proc. Int. Conf. Wireless Commun. Signal Process. (WCSP)*, 2018.

[8] D. G. M. Mitchell and E. Rosnes, "Edge spreading design of high rate array-based SC-LDPC codes," in *Proc. IEEE Int. Symp. Inf. Theory (ISIT)*, 2017.

[9] M. Battaglioni, M. Baldi, F. Chiaraluce, and D. G. M. Mitchell, "Efficient search and elimination of harmful objects in optimized QC SC-LDPC codes," *arXiv preprint arXiv:1904.07158*, 2019.

[10] H. Esfahanizadeh, A. Hareedy, and L. Dolecek, "A unified spatially coupled code design: Threshold, cycles, and locality," *arXiv preprint arXiv:2203.02052*, 2022.

[11] A. Hareedy, H. Esfahanizadeh, and L. Dolecek, "Adapt or regress: Finite-length design of spatially coupled codes," *arXiv preprint arXiv:2509.21112*, 2025.

[12] Y. Ren *et al.*, "Edge-spreading raptor-like LDPC codes for 6G," *arXiv preprint arXiv:2410.16875*, 2024.

[13] A. Tanrıkulu, M. Yıldırım, and A. Hareedy, "An MCMC method for efficient finite-length LDPC code design," *arXiv preprint arXiv:2504.16071*, 2025.

[14] I. Bello, H. Pham, Q. V. Le, M. Norouzi, and S. Bengio, "Neural combinatorial optimization with reinforcement learning," *arXiv preprint arXiv:1611.09940*, 2016.

[15] R. J. Williams, "Simple statistical gradient-following algorithms for connectionist reinforcement learning," *Machine Learning*, vol. 8, no. 3–4, pp. 229–256, 1992.

[16] A. R. Iyengar, M. Papaleo, P. H. Siegel, J. K. Wolf, A. Vanelli-Coralli, and G. E. Corazza, "Windowed decoding of spatially coupled codes," in *Proc. IEEE Int. Symp. Inf. Theory (ISIT)*, 2011.

[17] G. Liva and M. Chiani, "Protograph LDPC codes design based on EXIT analysis," in *Proc. IEEE Global Telecommun. Conf. (GLOBECOM)*, 2007.

[18] M. Stinner and P. M. Olmos, "Analyzing finite-length protograph-based spatially coupled LDPC codes," *arXiv preprint arXiv:1401.8090*, 2014.

[19] M. P. C. Fossorier, "Quasi-cyclic low-density parity-check codes from circulant permutation matrices," *IEEE Trans. Inf. Theory*, vol. 50, no. 8, pp. 1788–1793, 2004.

[20] T. Tian, C. R. Jones, J. D. Villasenor, and R. D. Wesel, "Selective avoidance of cycles in irregular LDPC code construction," *IEEE Trans. Commun.*, vol. 52, no. 8, pp. 1242–1247, 2004.